



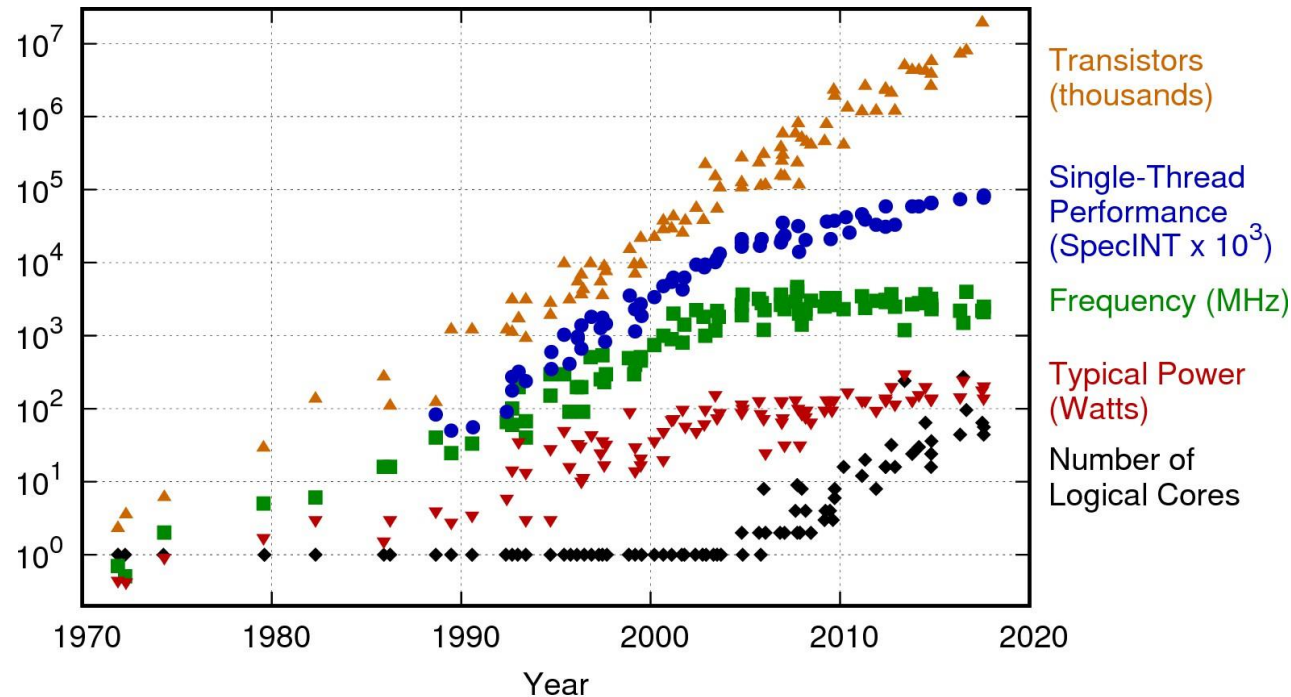
CS683: Advanced Computer Architecture

Lecture 12: Cache Coherence

<https://www.cse.iitb.ac.in/~biswa/courses/>

<https://www.cse.iitb.ac.in/~biswa/>

Shift to Multicore



Original data up to the year 2010 collected and plotted by M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, and C. Batten

New plot and data collected for 2010-2017 by K. Rupp [<https://www.karlsruhp.net/2018/02/42-years-of-microprocessor-trend-data/>]

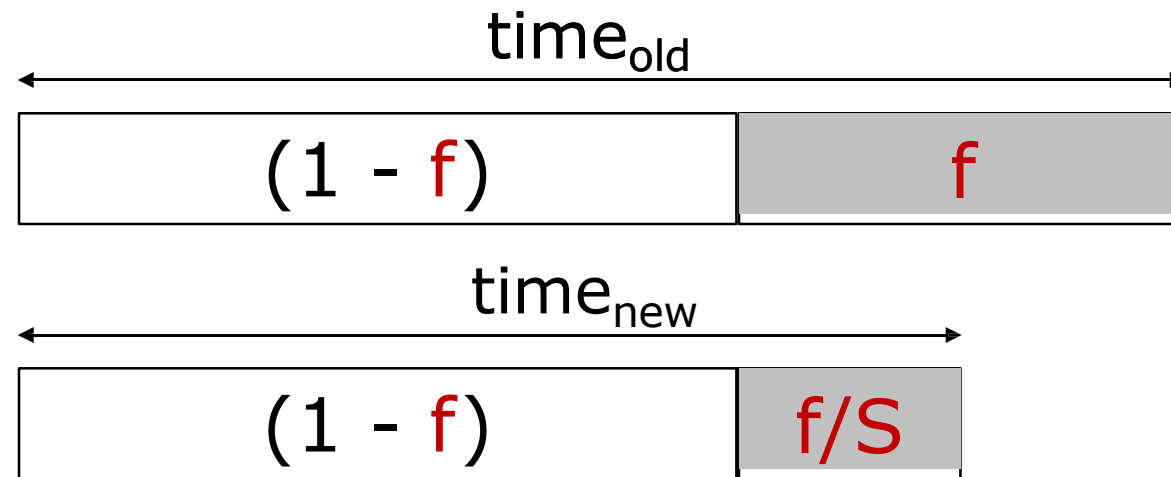
- Since 2005, improvements in system performance mainly due to increasing cores per chip
- Why? **Technology scaling**
Limited instruction-level parallelism
Advanced Computer Architecture

Remember Amdahl's Law

- Speedup = $\text{time}_{\text{without enhancement}} / \text{time}_{\text{with enhancement}}$
- Suppose an enhancement speeds up a fraction f of a task by a factor of S

$$\text{time}_{\text{new}} = \text{time}_{\text{old}} \cdot ((1-f) + f/S)$$

$$S_{\text{overall}} = 1 / ((1-f) + f/S)$$



Some reality check

- Say you write a program that can do 90% of the work in parallel, but the other 10% is sequential
- What is the maximum speedup you can get by running on a multicore machine?

$$S_{\text{overall}} = 1 / ((1-f) + f/S)$$

$$f = 0.9, S = \infty \rightarrow S_{\text{overall}} = 10$$

What f do you need to use a 1000-core machine well?

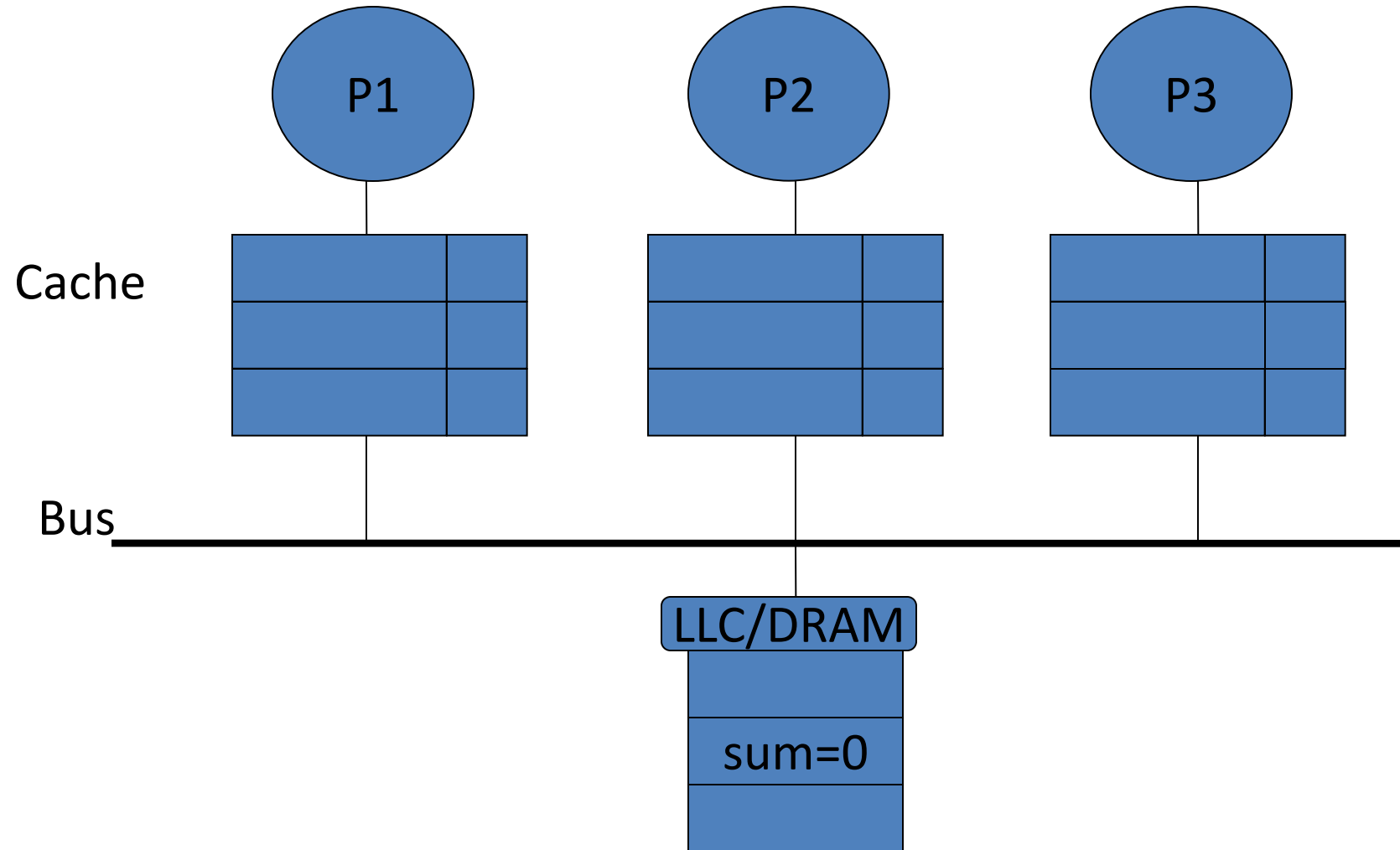
Parallel programs and communication models

- **Shared memory:**
 - Single address space
 - Implicit communication by reading/writing memory
 - Data
 - Control (semaphores, locks, barriers, ...)
 - Low-level programming model: threads
- **Message passing:**
 - Separate address spaces
 - Explicit communication by send/rcv messages
 - Data
 - Control (blocking msgs, barriers, ...)
 - Low-level programming model:
 - processes + inter-process communication (e.g., MPI)

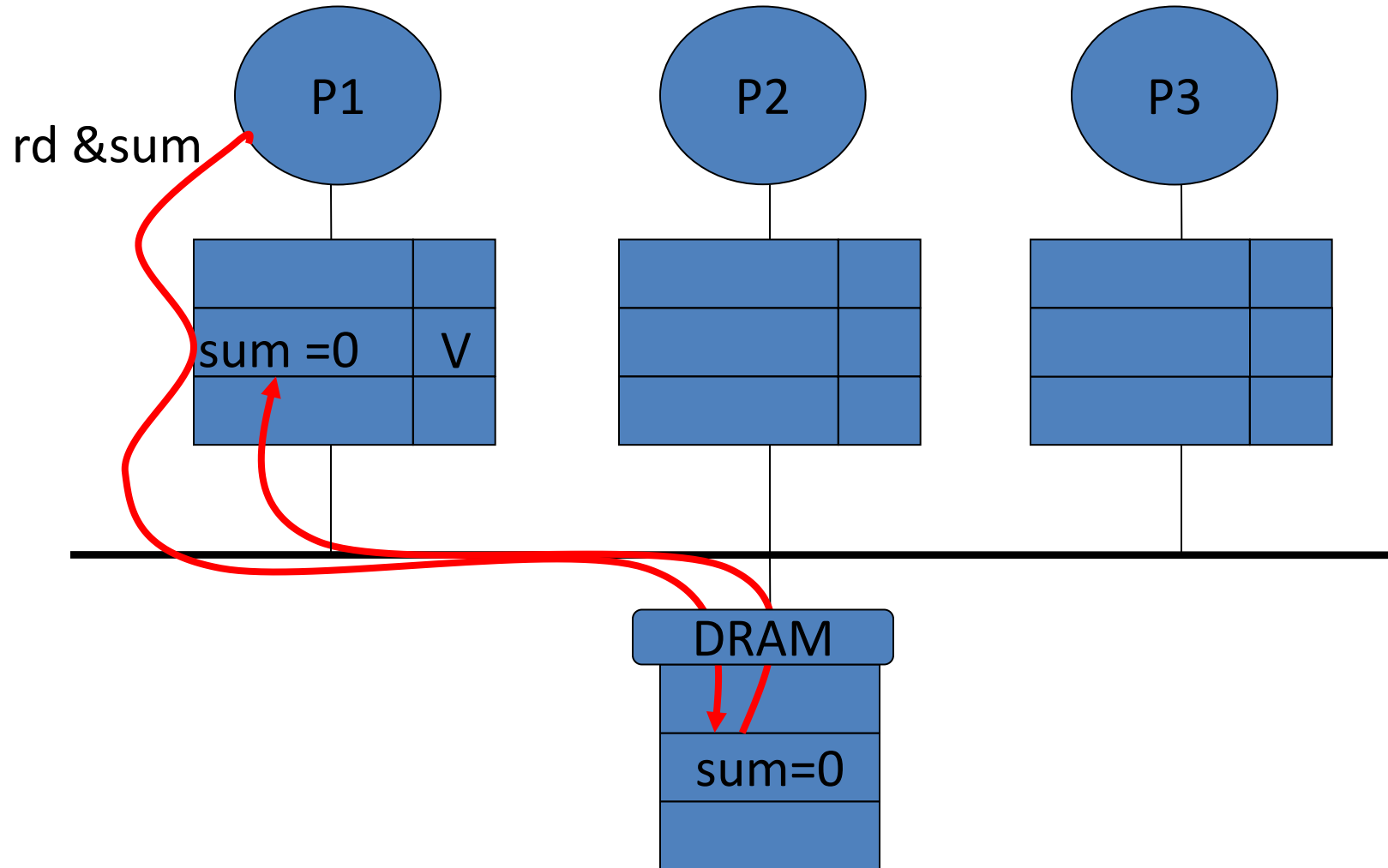
Coherence vs Consistency

- Shared memory systems:
 - Have **multiple private caches** for performance reasons
 - Need to provide the illusion of a single shared memory
- Intuition: A read should return the most recently written value
 - What is “most recent”?
- Formally:
 - Coherence: What values can a read return?
 - Concerns reads/writes to a single memory location
 - Consistency: When do writes become visible to reads?
 - Concerns reads/writes to multiple memory locations

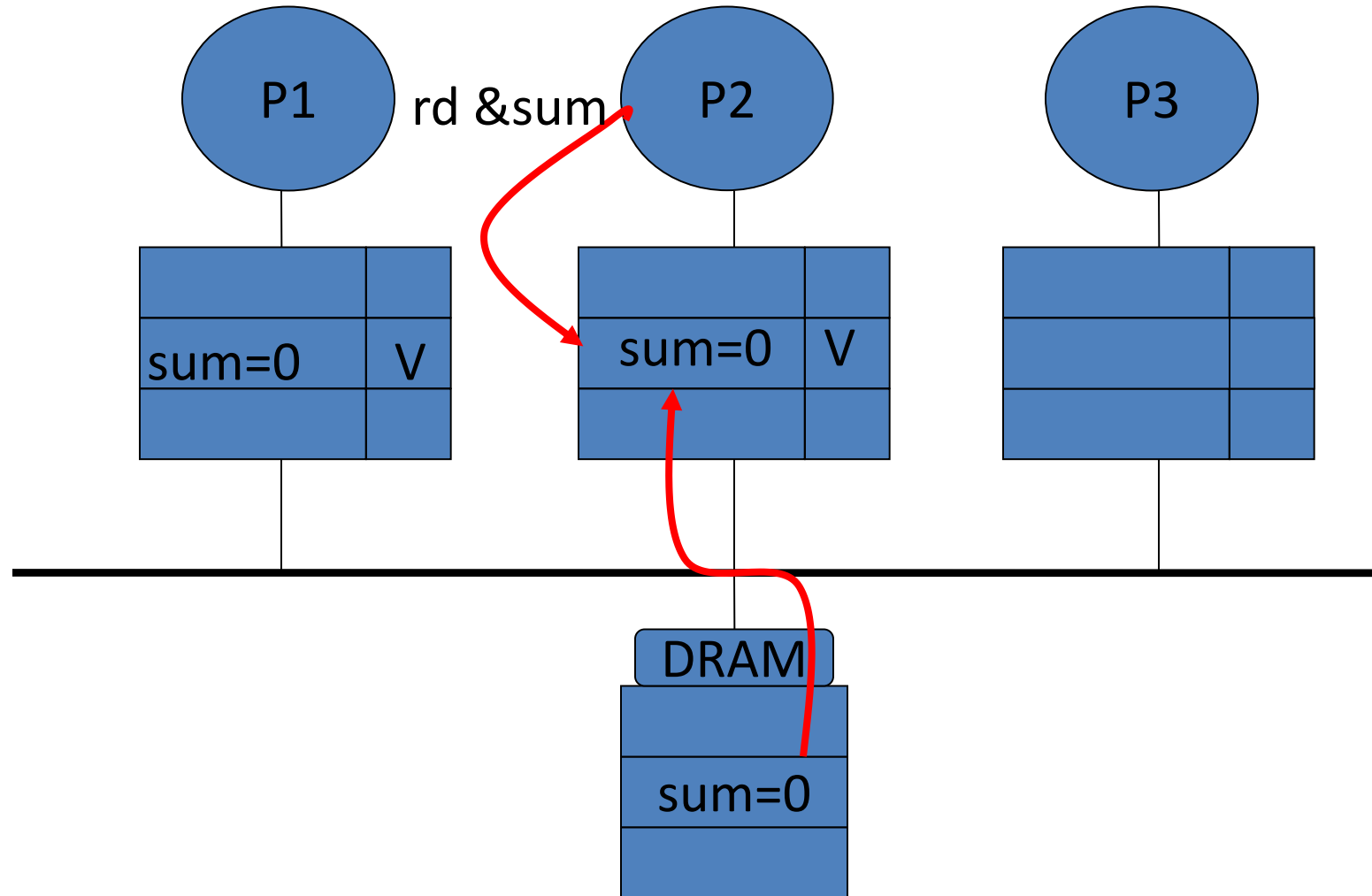
Cache Coherence



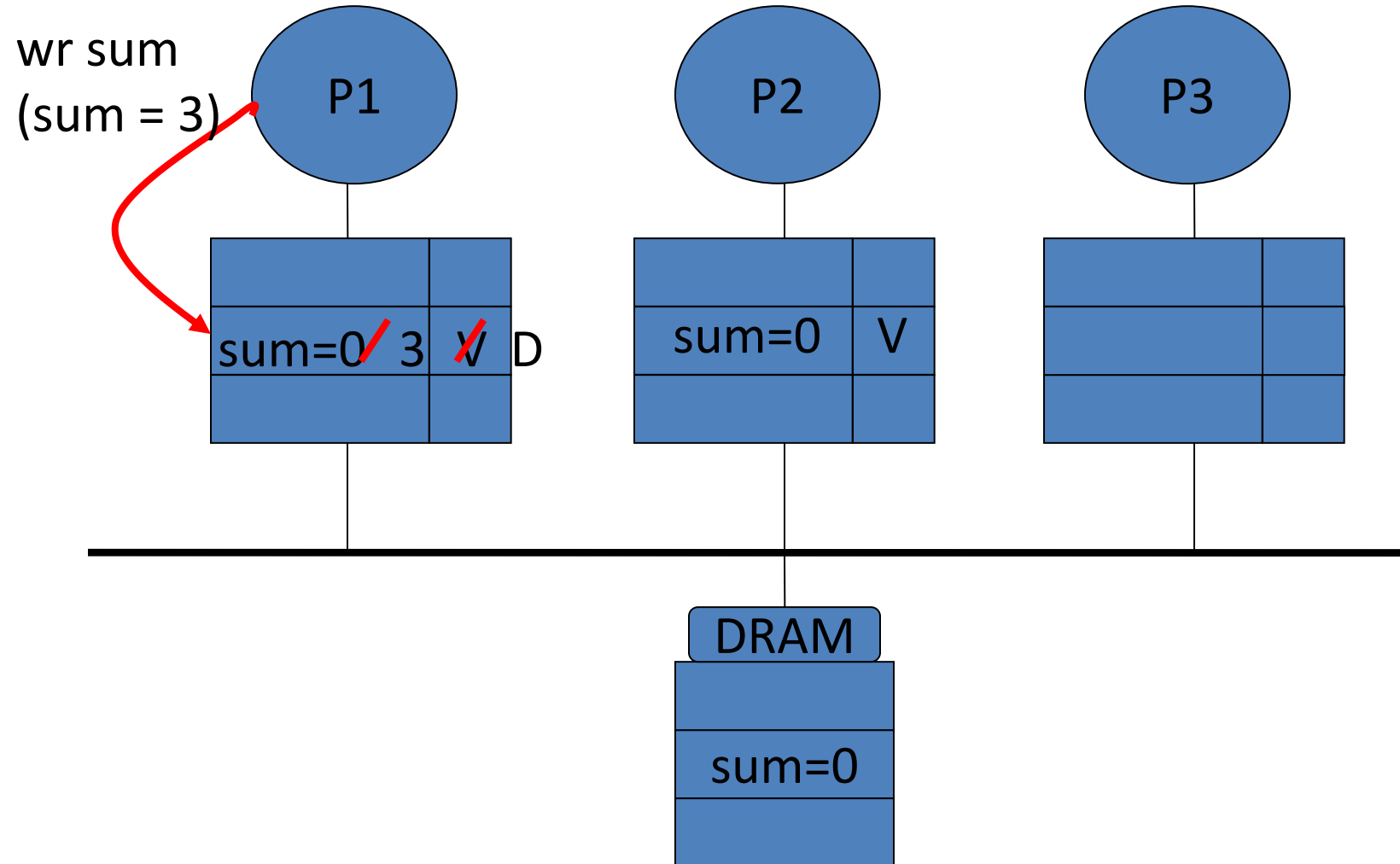
Cache Coherence



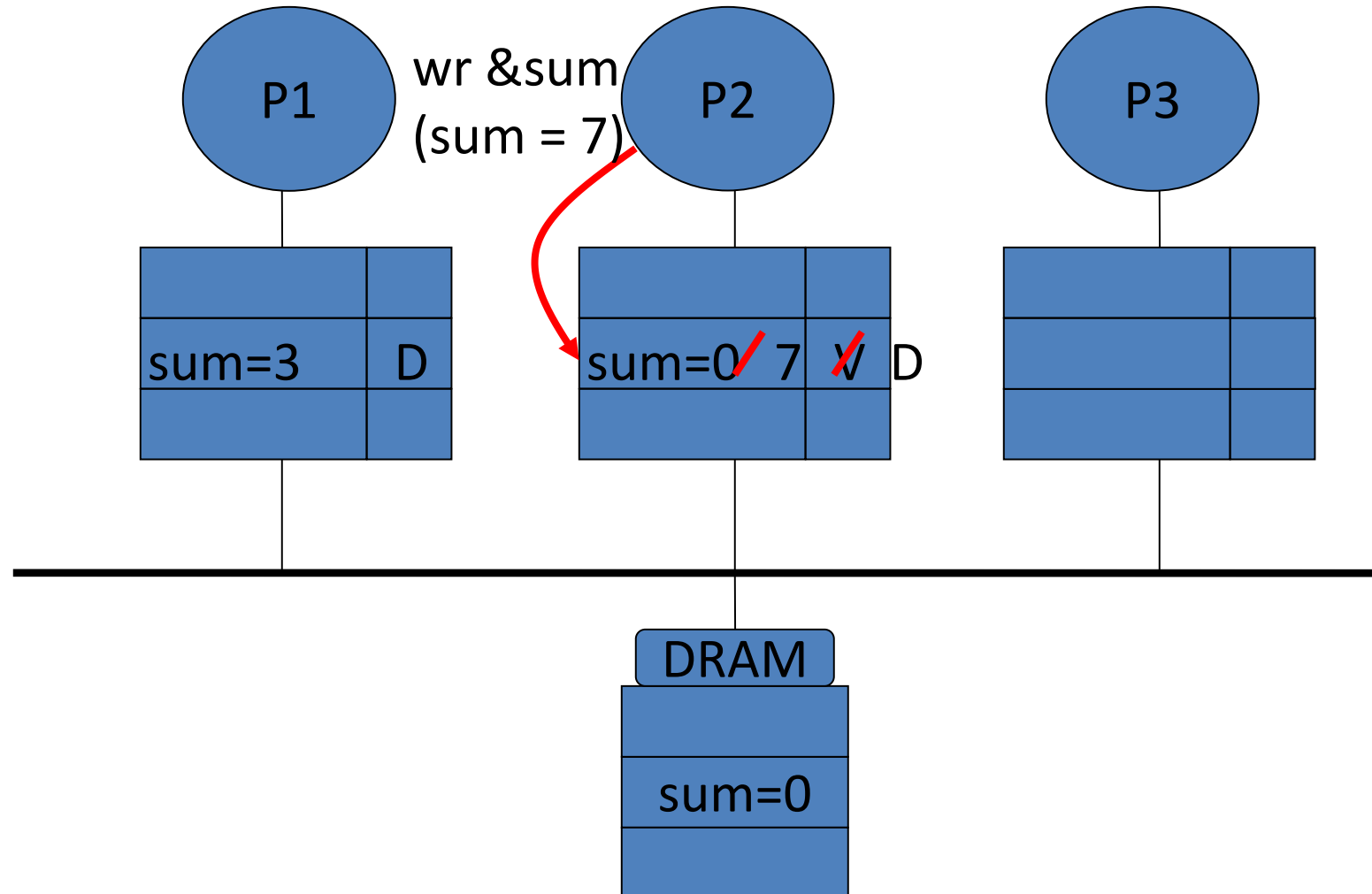
Cache Coherence



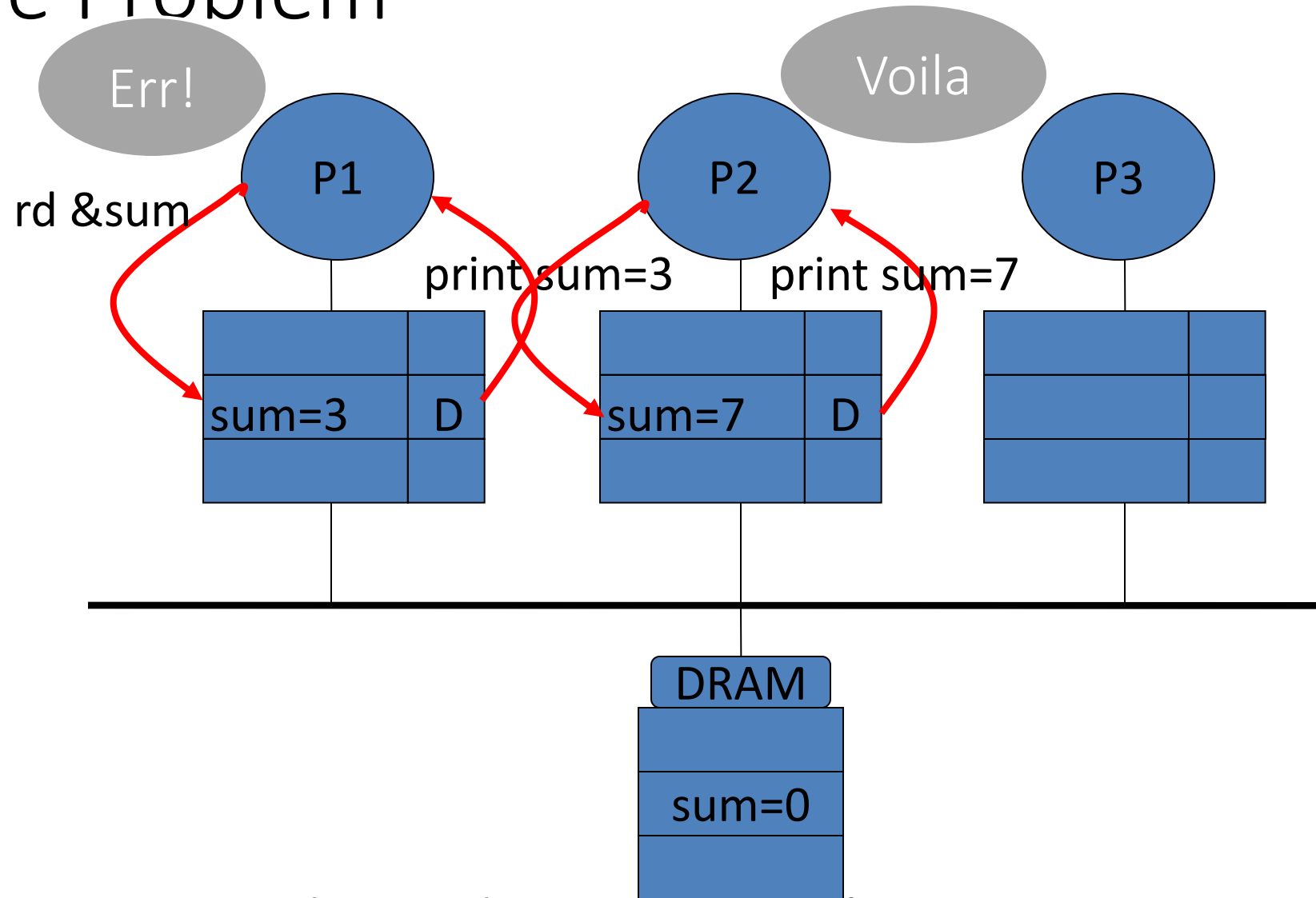
Cache Coherence



Cache Coherence



The Problem



The Problem

If multiple cores cache the same block, how do they ensure they all see a consistent state?

Solution:

1. **Write Propagation:** All writes eventually become visible to other cores.
2. **Write Serialization:** All cores see write to a cache line in same order.

Need a protocol to ensure (1) and (2) called *cache coherence protocol*

Invalidate/Update

Invalidate –

- ❑ Write to a cache line, and simultaneously broadcast invalidation of address to all others
- ❑ Other cores clear/invalidate their cache lines



Update –

- ❑ Write to a cache line, and simultaneously broadcast written data to all others
- ❑ Other cores update their caches if data was present



A Simple MSI protocol

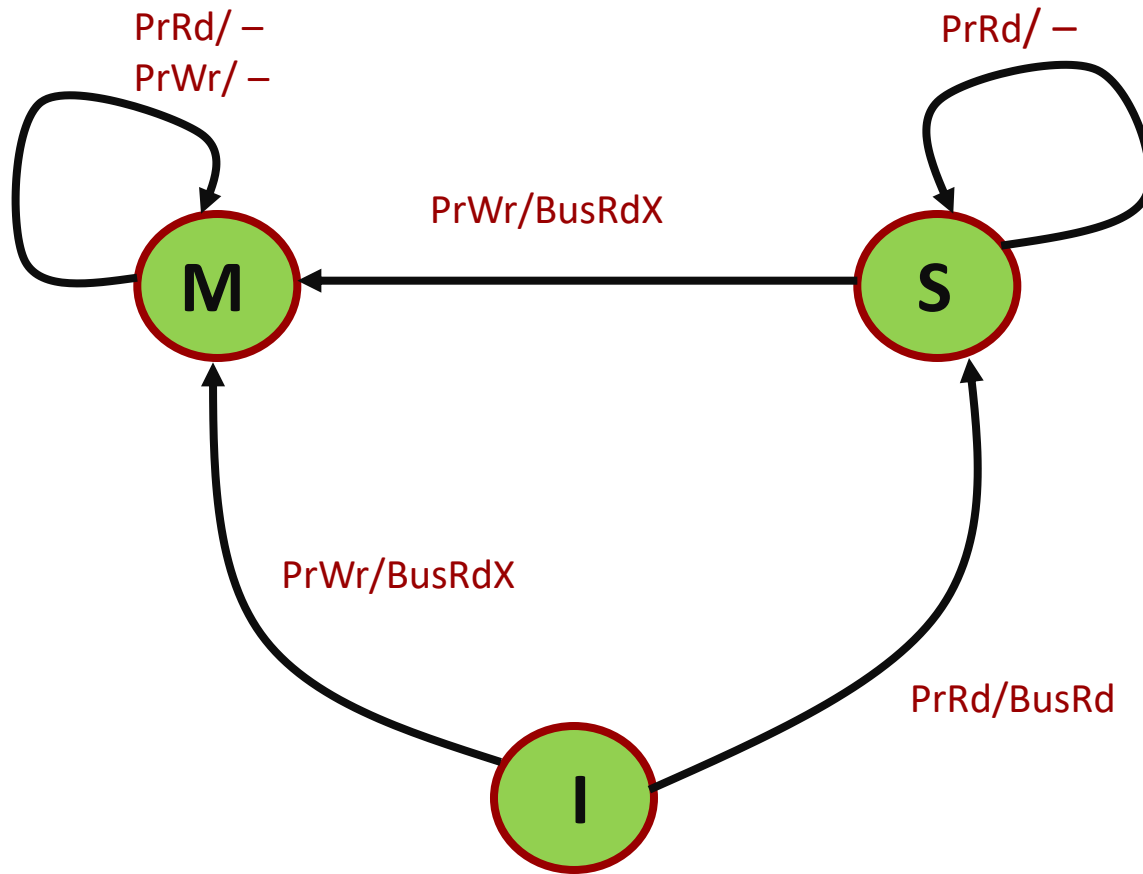
Extend single valid bit per line to three states:

- ❑ **M**(odified): only one cache, memory not updated.
- ❑ **S**(hared): one or more caches, and memory copy is up-to-date
- ❑ **I**(nvalid): not present
- ❑ These bits will be at the tag store as before

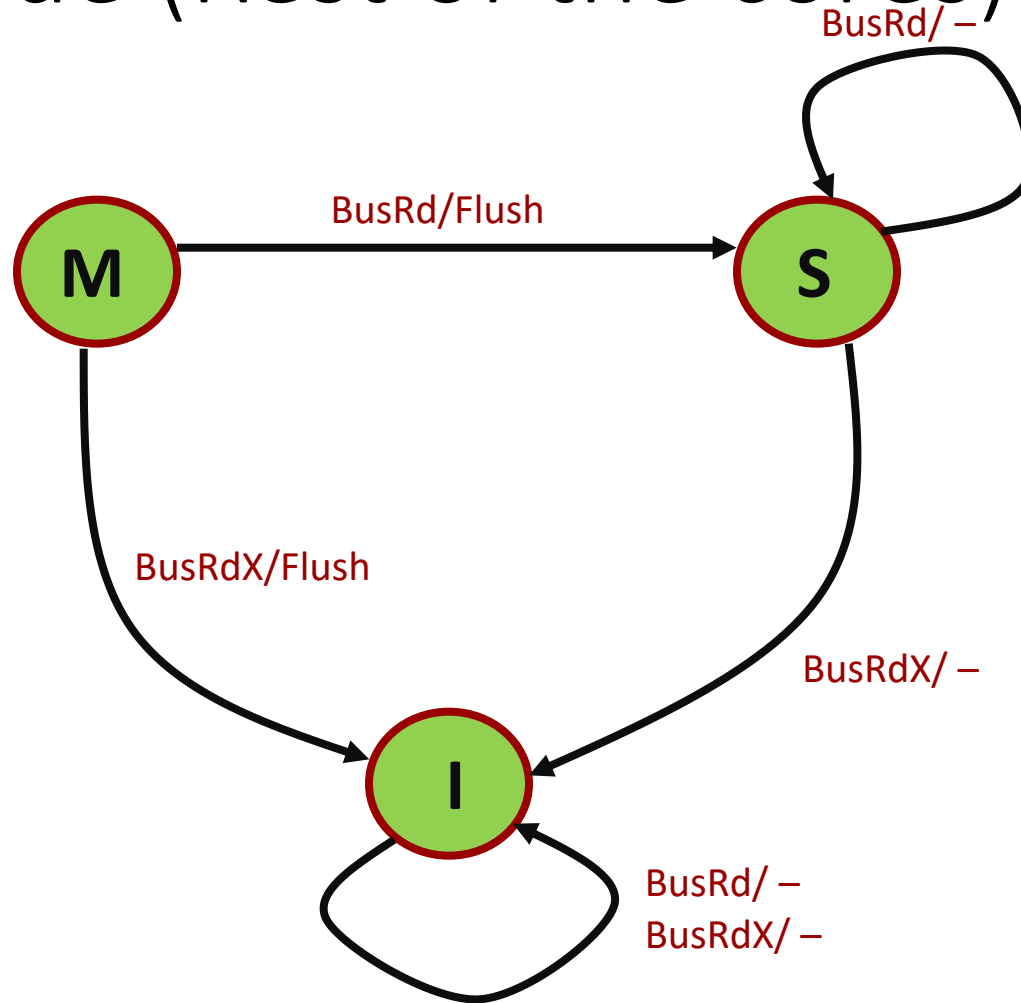
Read - *Read* request on bus, transitions to **S**

Write - *ReadEx* request, transitions to **M** state

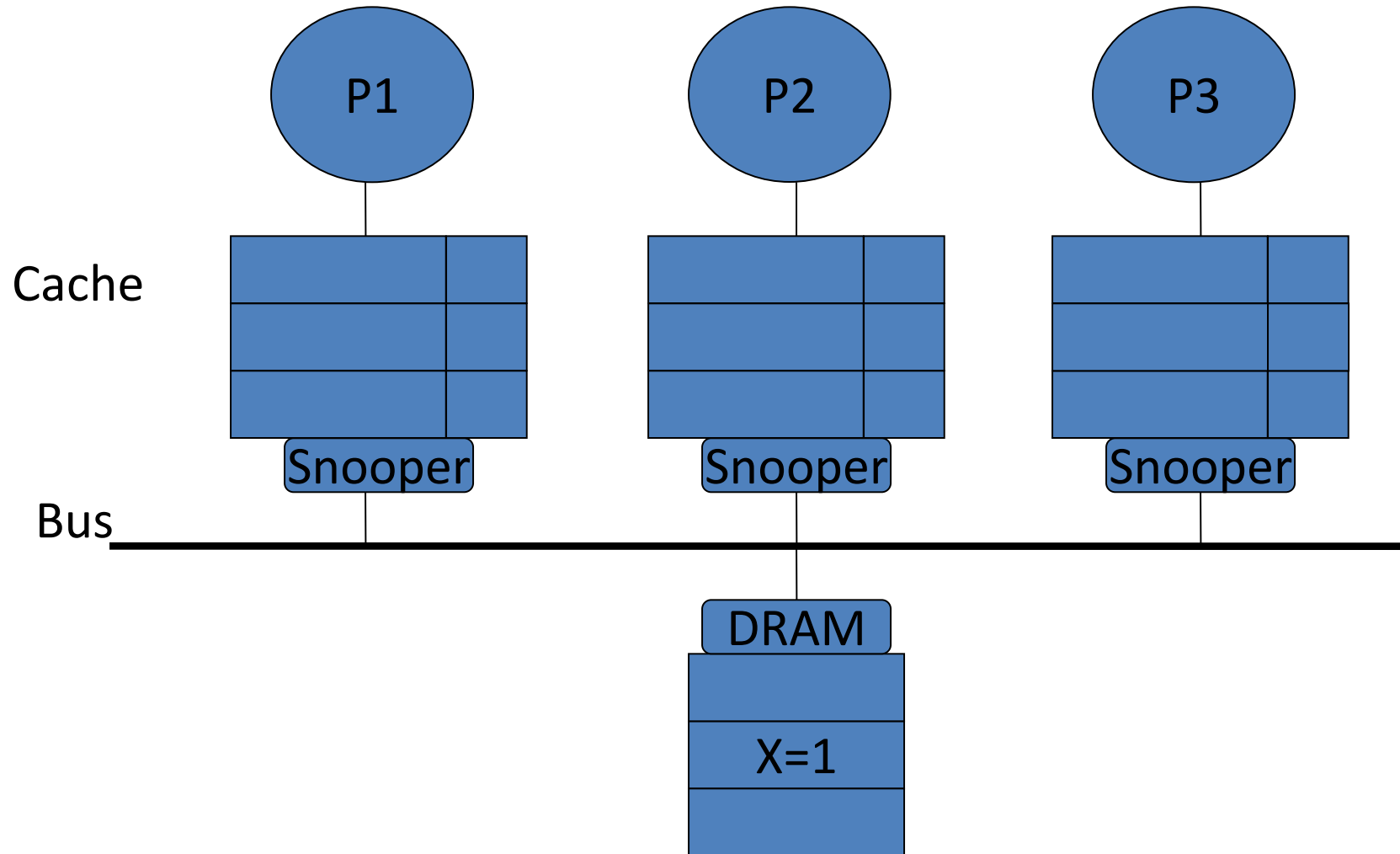
State-transitions: Processor side (Master core)

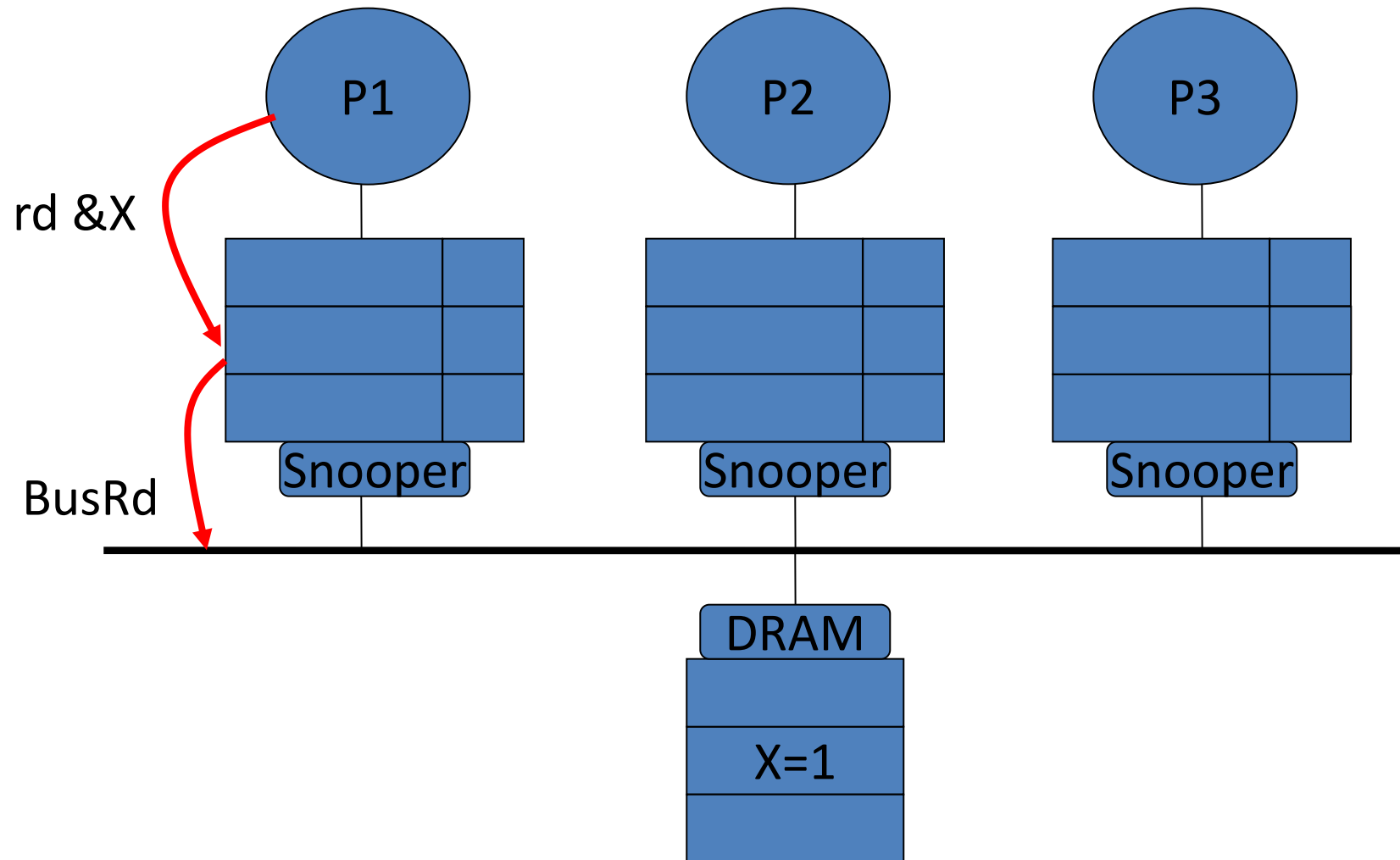


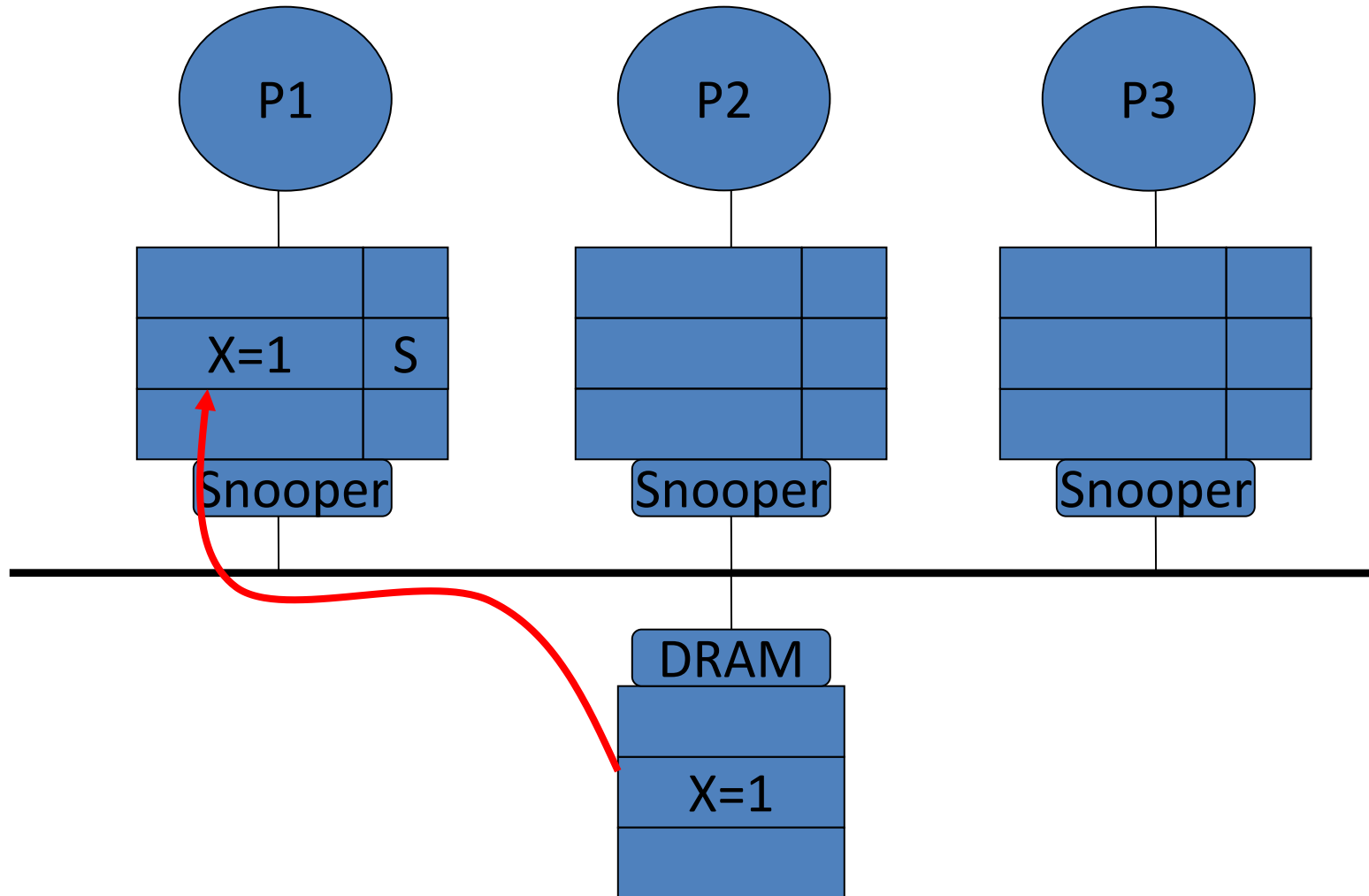
Bus-side (Rest of the cores)

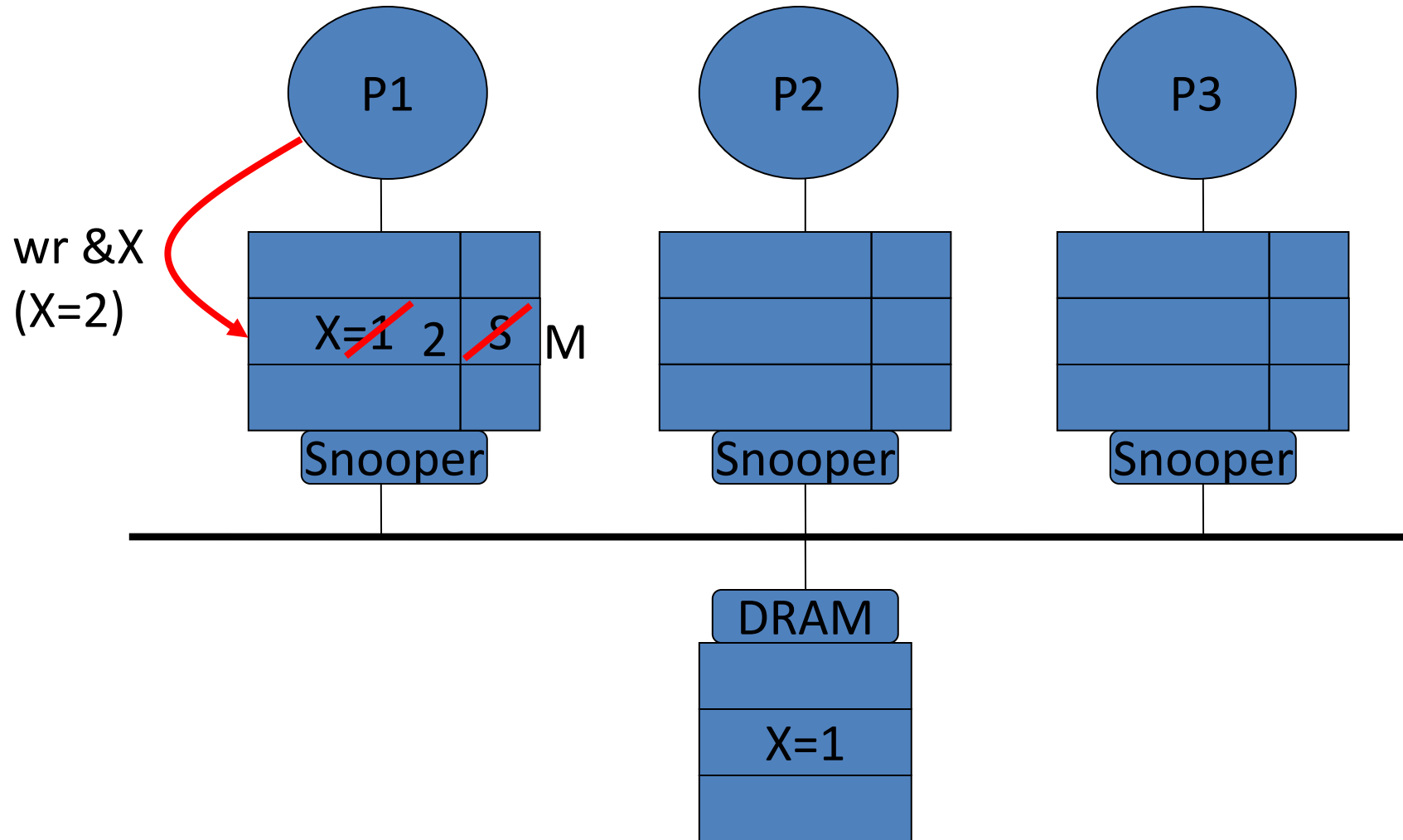


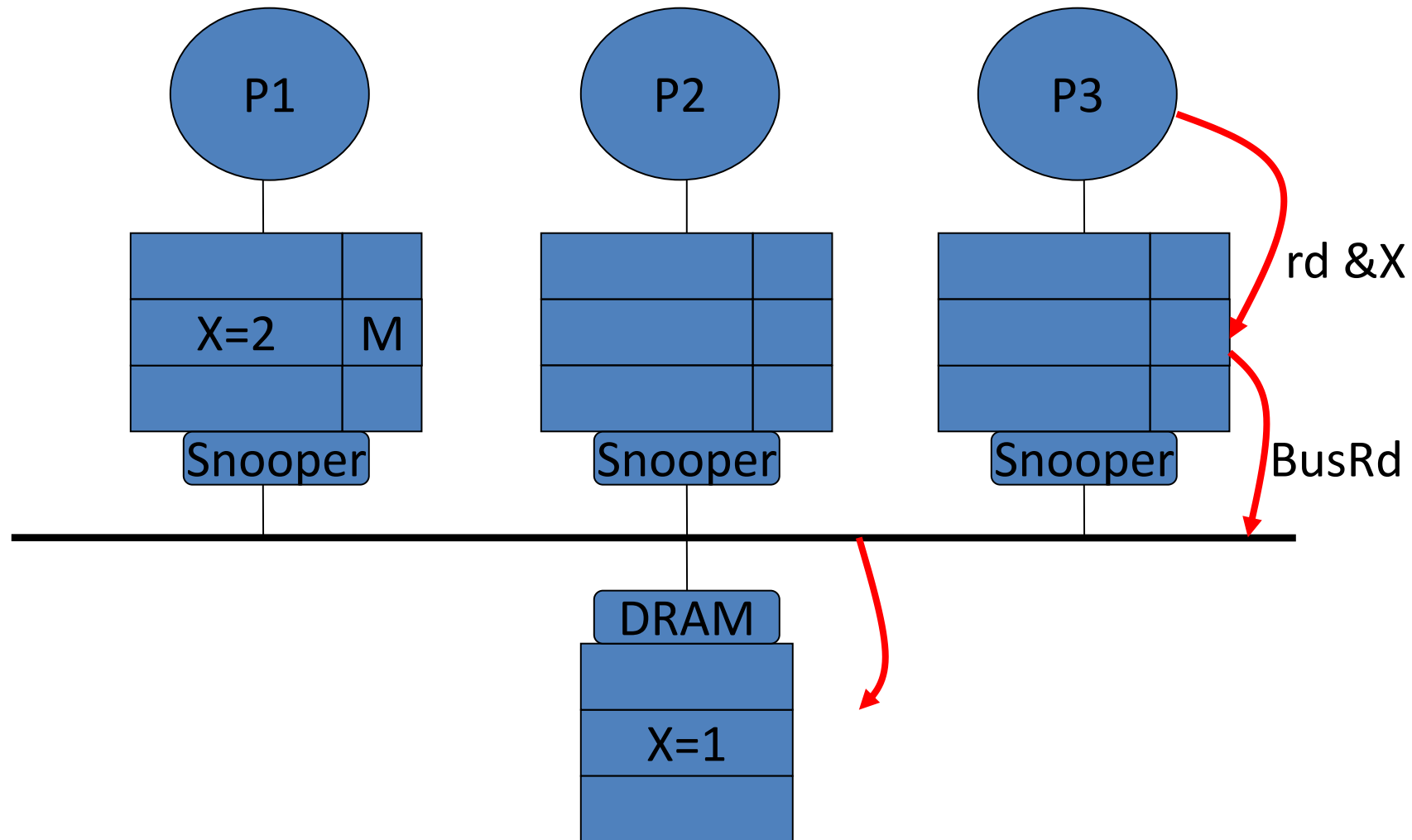
MSI Protocol

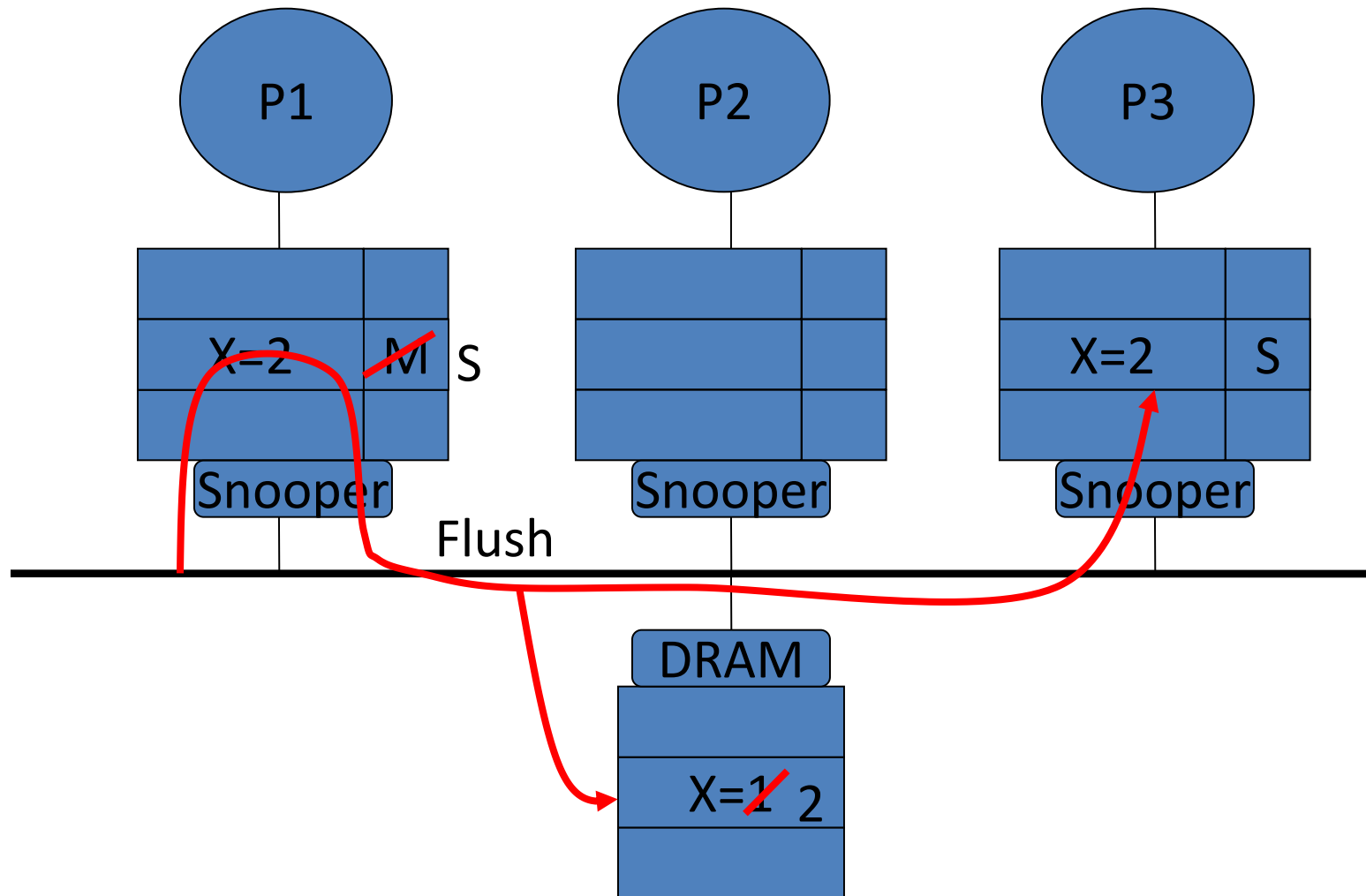




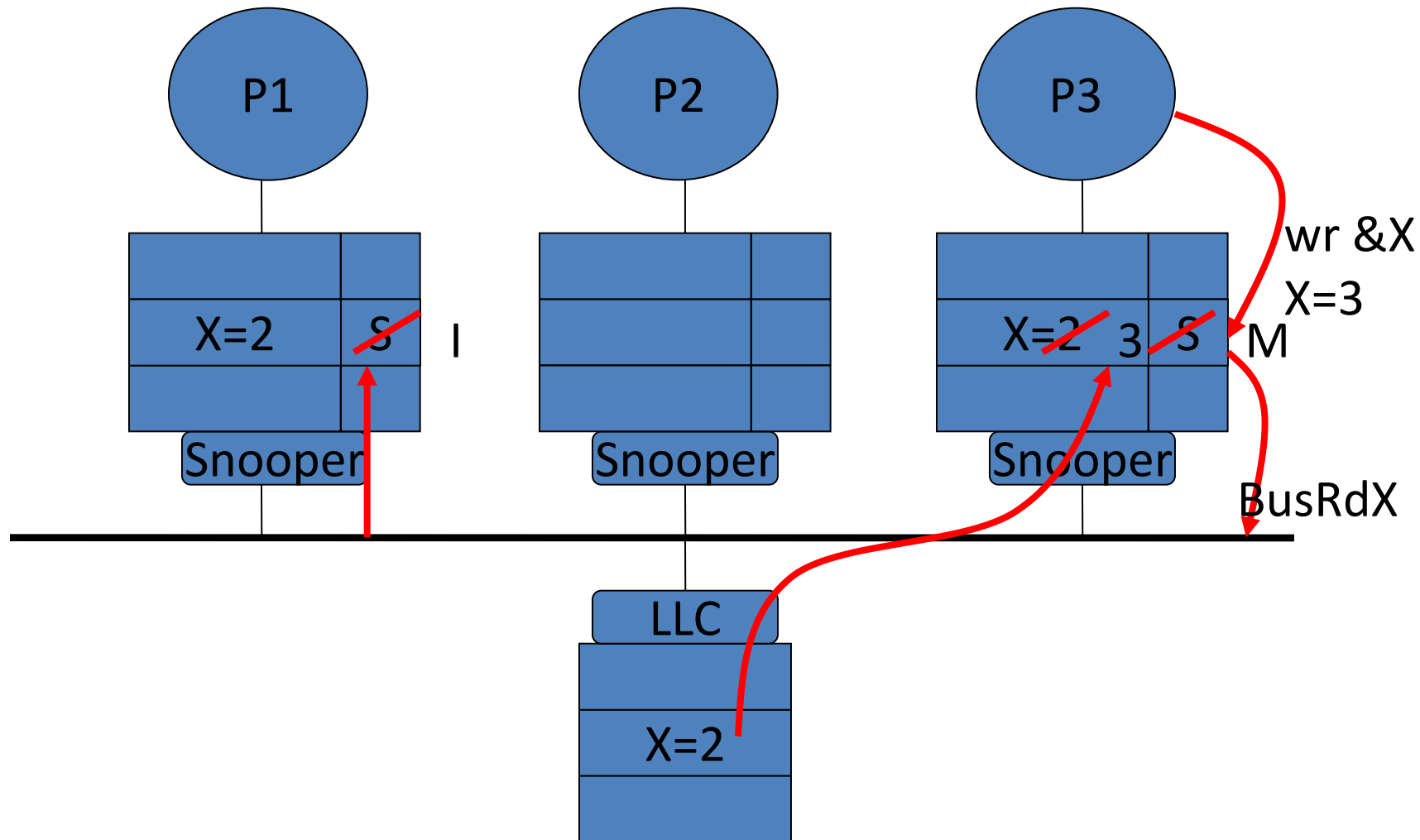


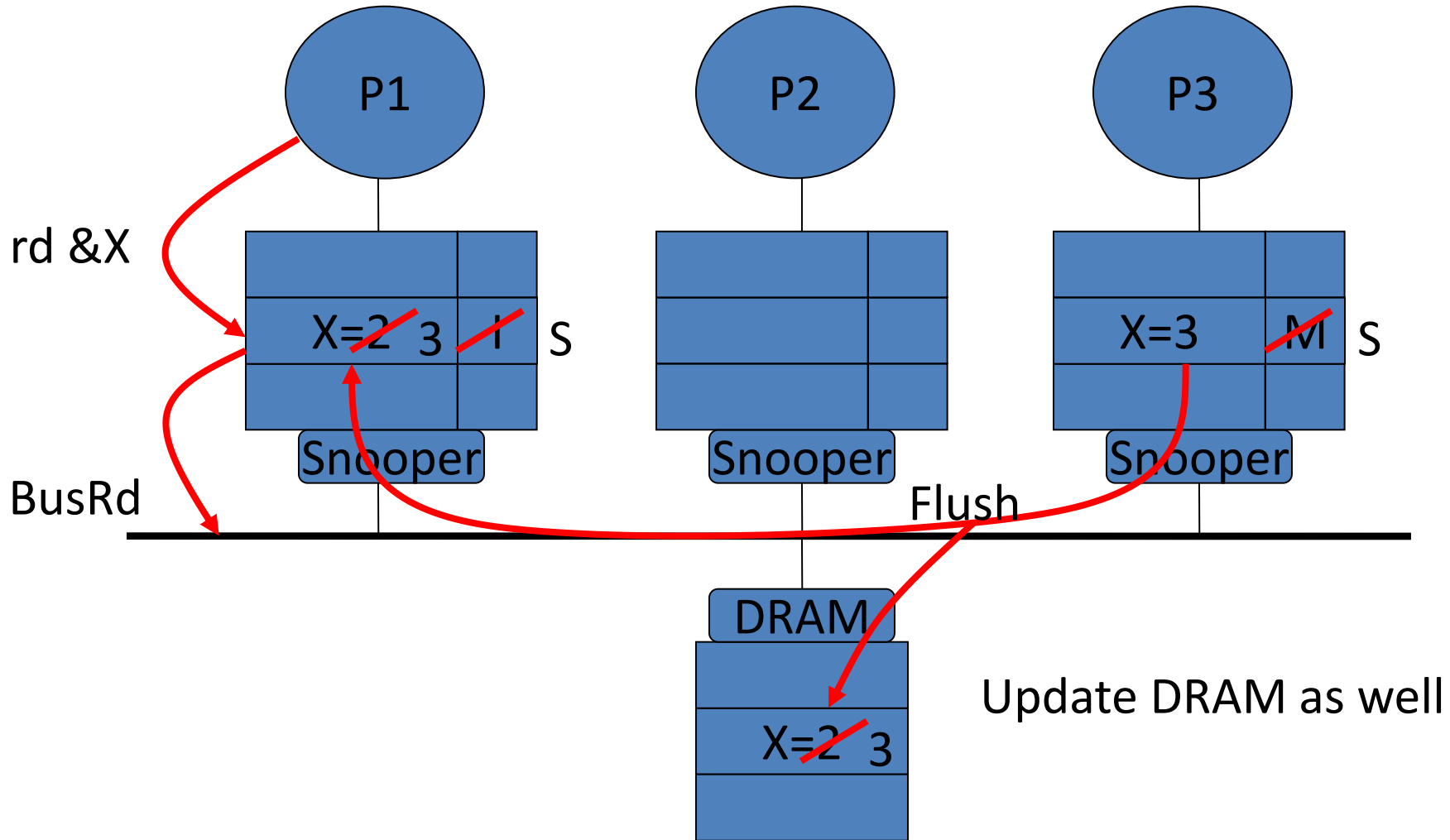


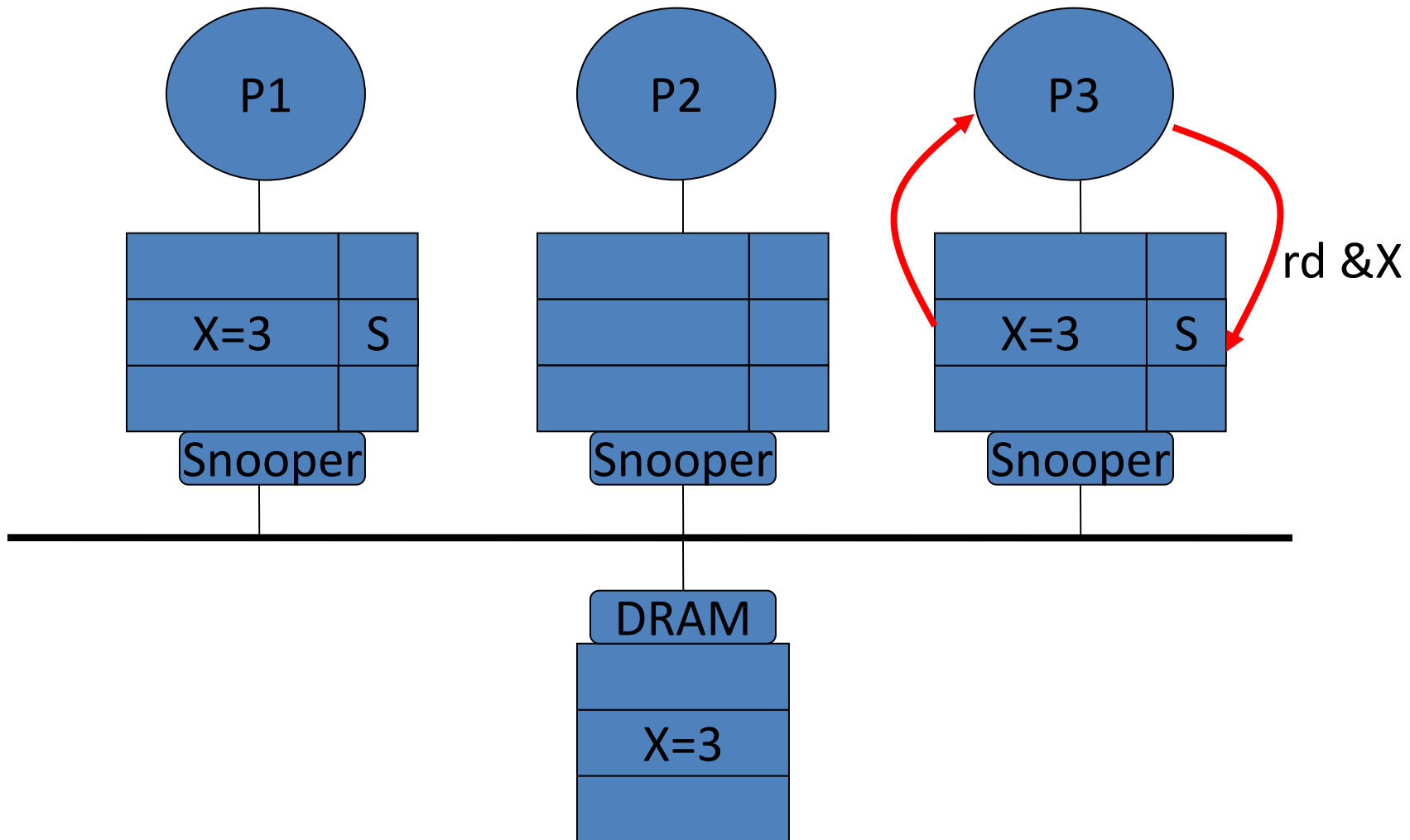




LLC/DRAM as the coherence point







Summary

Proc Action	State P1	State P2	State P3	Bus Action	Data From
R1	-	-	-	BusRd	Mem
W1	-	-	-	BusRdX	Mem
R3	-	-	-	BusRd	P1 cache
W3	-	-	-	BusRdX	Mem
R1	-	-	-	BusRd	P3 cache
R3	-	-	-	-	-
R2	-	-	-	BusRd	Mem

Summary

Proc Action	State P1	State P2	State P3	Bus Action	Data From
R1	S	-	-	BusRd	Mem
W1	M	-	-	BusRdX	Mem
R3	S	-	S	BusRd	P1 cache
W3	I	-	M	BusRdX	Mem
R1	S	-	S	BusRd	P3 cache
R3	S	-	S	-	-
R2	S	S	S	BusRd	Mem